Question 1

Question 1 - What have You Done? [30%]

One day, handsome Jeff finished grading n assignment scripts. He sorted the scripts in ascending order of UID (assume all UIDs are unique). To inspect his work, Ben read k scripts from the top, and put them back at the bottom of the pile without changing the order of the k scripts.

Suppose the UID of the n scripts are stored in list A as integers with the current placing order. The first element in A represents UID of the script on the top. To arrange the papers back in order, Jeff needs to find out the value of k.

You may assume that n is in power of 2, and $0 \leq k \leq n-1$.

Example:
For input $A_1 = [43, 60, 71, 3, 14, 23, 26]$ and $n = 7$, the output will be $k = 4$. Ben has read scripts with UID 3, 14, 23, 26.

Your Tasks:

(a) [10%] Jeff has written the function reOrder1(A, n) to solve the problem. Explain briefly how his code solve the problem and analyze it's worst case time complexity.

Algorithm 1.1 Pseudocode of reOrder1

```
function reOrder1(A, n)

    k = 0
    for (i = 0 to n − 2) do
        if (A[i] > A[i + 1]) then
            k = n − i − 1
        end if
    end for

    return k

end function
```

(b) [15%] If n is large, it would be a nightmare to run Jeff's algorithm by human efforts. Write the pseudocode of another algorithm reOrder2(A, n) to solve the problem. The algorithm must be asymptotically faster than reOrder1(A, n).

Briefly explain what the code does, and analyze the algorithm's worst case time complexity.

(a)

The function checks from the start of the list A if the number is greater than the next number. If it is, then k is number of elements after the current indice, given by $n - i - 1$ where i is the current indice.

It's worst time case complexity is $O(n)$ for when k is 0.

(b)

```
function REORDER2(A, n)
    if (n <= 1 or A[0] < A[n-1])
        return 0
    left = 0
    right = n - 1
    while (left + 1 < right)
        middle = (left + right) / 2
        if (A[middle] < A[n-1])
            right = middle
        else
            left = middle
    return n - right
```

The worst time case complexity is $O(\log n)$, as the search space is halved each time.

The code returns 0 if the list is sorted in ascending order, as there is no read scripts, otherwise, it works by checking if the middle element is less than the last element. If it is, then the pivot point is in the left half of the list (inclusive), otherwise, it's in the right half. This is because the pivot point is the smallest element in the list. We repeat this process until the search space is reduced to 2 elements, at which point we return the index of the smaller element.

Question 2

Question 2 - Linked List [30%]

Implement a linked list and solve the following sub-problems with your implementation. Feel free to select any programming language you prefer.

(a) [15%] Merging: Implement a function that takes two sorted linked lists and merges them into a single sorted linked list.

(b) [15%] Cycle detection: A cycle in a linked list occurs when a node's next pointer points back to a previous node, creating a loop, while the beginning of a cycle is the first node in that loop where traversal starts repeating.

Implement a function that detects if there is a cycle in the linked list. If a cycle exists, it returns the node where the cycle begins.

```
class Node {
  constructor(data) {
    this.data = data;
    this.next = null;
  }
}

class LinkedList {
  constructor(data) {
    if (data) {
      this.head = new Node(data);
    }
    this.head = null;
  }

  push(data) {
    const newNode = new Node(data);
    const head = this.head;
    if (head) {
      let current = head;
      while (current.next !== null) {
        current = current.next;
      }
      current.next = newNode;
    } else {
      this.head = newNode;
    }
  }

  print() {
    let current = this.head;
    while (current !== null) {
      console.log(current.data);
      current = current.next;
    }
```

```
  }
}

// Question 2a
function mergeAndSort(l1, l2) {
  const merged = new LinkedList();
  let current1 = l1.head;
  let current2 = l2.head;
  while (current1 !== null || current2 !== null) {
    if (current1 === null) {
      merged.push(current2.data);
      current2 = current2.next;
    } else if (current2 === null) {
      merged.push(current1.data);
      current1 = current1.next;
    } else if (current1.data < current2.data) {
      merged.push(current1.data);
      current1 = current1.next;
    } else {
      merged.push(current2.data);
      current2 = current2.next;
    }
  }
  return merged;
}

// Question 2b
function getCycledNode(l) {
  let current = l.head;
  let traversed = [];
  while (current !== null) {
    const next = current.next;
    if (next === null) break;
    if (traversed.includes(next)) return current;
    traversed.push(current);
    current = next;
  }
  return null;
}
```

Question 3

Question 3 - Maximum Frequency Stack [20%]

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Specifically, for a *FreqStack* class: *FreqStack*() constructs an empty frequency stack (called *fstack*). *fstack.push*(*val*) pushes an integer *val* onto the top of the stack. *fstack.pop*() removes and returns the most frequent element in the stack. If there are multiple different elements of the same highest frequency, the element closest to the stack's top is removed and returned.

Suppose at most N elements will be inserted into *fstack* and each *val* is smaller than 10.

Example:

```
FreqStack fstack = new FreqStack();
fstack.push(5); // The stack is [5]
fstack.push(7); // The stack is [5,7]
fstack.push(5); // The stack is [5,7,5]
fstack.push(7); // The stack is [5,7,5,7]
fstack.push(4); // The stack is [5,7,5,7,4]
fstack.push(5); // The stack is [5,7,5,7,4,5]
fstack.pop(); // return 5, as 5 is the most frequent. The stack becomes [5,7,5,7,4].
fstack.pop(); // return 7, as 5 and 7 are the most frequent, but 7 is closest to the
    top. The stack becomes [5,7,5,4].
fstack.pop(); // return 5, as 5 is the most frequent. The stack becomes [5,7,4].
fstack.pop(); // return 4, as 4, 5 and 7 are the most frequent, but 4 is closest to
    the top. The stack becomes [5,7].
```

Your Tasks:

(a) [15%] Implement the *FreqStack* class as described above.

(b) [5%] Analyse the worst-case time complexity of *fstack.push* and *fstack.pop* of your implementation.

(a)

```
class FreqStack {
  constructor() {
    this.f = {};
    this.n = [];
  }

  push(x) {
    this.f[x] = (this.f[x] || 0) + 1;
    this.n.push(x);
```

```
  }

  pop() {
    let toPop = null;
    for (let i = 0; i <= this.n.length - 1; i++) {
      const x = this.n[i];
      const toPopValue = this.n[toPop] || 0;
      const toPopFreq = toPop ? this.f[toPopValue] : 0;
      const xFreq = this.f[x] || 0;
      if (xFreq >= toPopFreq) {
        toPop = i;
      }
    }
    const toPopValue = this.n[toPop];
    this.f[toPopValue]--;
    console.log("popping", toPopValue);
    this.n.splice(toPop, 1);
  }
}
```

(b)

The worst case time complexity of push is O(1) and the worst case time complexity of pop is O(n).

Question 4

Question 4 - Hash Table [20%]

Insert the keys $\{17, 94, 86, 22, 98, 79, 54, 38\}$ into the following hash tables.

For each case, calculate the average number of slots inspected for an unsuccessful search after the keys are inserted (An empty slot or a NIL pointer access should also be counted as one inspection.). Assume that each slot in a hash table has equal chance to be accessed at the first probe in the search process.

(a) [5%] A hash table of size 7 with a single hash function of $h(x) = x \bmod 7$. Collisions are handled by chaining.

(b) [7%] A hash table of size 11 with a single hash function of $h(x) = x \bmod 11$. Collisions are handled by open addressing with linear probing, using the function $f(i) = 3i$.

(c) [8%] A hash table of size 11 with double hashing using the probe sequence $h(k, i) = (h_1(k) + i\ h_2(k)) \bmod 11$, where $h_1(x) = x \bmod 11$ and $h_2(x) = 2 - (k \bmod 2)$.

(a)

$m = 7, \quad h(k) = k \mod 7$ with chaining.

Key	17	94	86	22	98	79	54	38
Hash	3	3	2	1	0	2	5	3
Slot	3[0]	3[1]	2[0]	1[0]	0[0]	2[1]	5[0]	3[2]

Average number of slots inspected for unsuccessful search: $\frac{8}{7}$

(b)

$m = 11, \quad h(k) = k \mod 11$ with linear probing $f(i) = 3i$.

Key	17	94	86	22	98	79	54	38
Hash	6	6	9	0	10	2	10	5
Slot	6	~~6~~ 9	~~9~~ 1	0	10	2	~~10~~ ~~2~~ 5	~~5~~ 8

Average number of slots inspected for unsuccessful search: $\frac{1}{1-\frac{8}{11}} = \frac{11}{3}$

(c)

$m = 11, \quad h(k) = k \mod 11$ with double hashing $f(i) = i \cdot h'(k), \quad h'(k) = 2 - (k \mod 2)$.

Key	17	94	86	22	98	79	54	38
Hash	6	6	9	0	10	2	10	5
Slot	6	~~6~~ 8	9	0	10	2	~~10~~ 1	5

Average number of slots inspected for unsuccessful search: $\frac{1}{1-\frac{8}{11}} = \frac{11}{3}$